Assessing Difficulty of Link Prediction on Benchmarking Knowledge Graphs

by

Lydia Klecan

A thesis submitted in partial fulfillment of the
requirements for the degree of
Master of Science
in Computing and Information Sciences

B. Thomas Golisano College of Computing and
Information Sciences
Rochester Institute of Technology

April 28, 2026

Assessing Difficulty of Link Prediction on Benchmarking Knowledge Graphs

---

APPROVED BY
SUPERVISING COMMITTEE

Carlos Rivero, Advisor

Zachary Butler, Reader

Matthew Fluet, Observer

Date: 4-28-2026

Assessing Difficulty of Link Prediction on Benchmarking Knowledge Graphs

by

Lydia Klecan

Submitted to the
B. Thomas Golisano College of Computing and Information Sciences
Department of Computer Science
in partial fulfillment of the requirements for the
Master of Science Degree
at the Rochester Institute of Technology

Abstract

Knowledge graphs represent real-world data in a directed graph format where two entities, connected by a predicate, represent one fact. Link prediction models predict new relationships using existing entities and predicates, and are trained using benchmarking knowledge graphs. Redundancies exist within benchmarking knowledge graphs that allegedly artificially improve the results of link prediction. It is assumed in the link prediction field that if a model scores well on a highly redundant benchmarking knowledge graph, it may not directly correlate to good performance on other more complex knowledge graphs.

This research creates new analysis methods and evaluation metrics for measuring redundancies in knowledge graphs. We use Horn rules to define five different redundancy types: near-duplicate, near-reverse, symmetric, transitive, and Cartesian product. Support and confidence of these rules quantify the redundancy. Using the quantified results, the two main goals are: (1) measuring levels of redundancy in benchmarking knowledge graphs and (2) offsetting link prediction results based on predicate-specific redundancies within knowledge graphs. Using a single metric to report levels (1) confirms high levels of redundancy in FB15k, WN18 and YAGO3-10, which are known to be highly redundant. High levels of redundancy are also seen for BioKG, which is a new finding. Offsetting link prediction results (2) was done by using predicate-specific redundancy values as weights for several metrics borrowed from the

information retrieval field: RR, $R@k$, and $BPM@k$. This method resulted in decreased values across each metric for FB15k, WN18RR, and YAGO3-10, indicating that redundancies artificially inflate link prediction scores on these knowledge graphs. However, Hetionet and NELL-995 show increased values across each metric, indicating that redundancies do not have the same impact on those knowledge graphs. Other knowledge graphs showed mixed results across different link prediction models and metrics. These results indicate that redundancies in benchmarking knowledge graphs may not have the same impact across different knowledge graphs, link prediction models, and evaluation metrics.

The new methods introduced to measure redundancy provide important insights to interpret behavior for link prediction. It becomes even more important to be aware of the level of redundancy in knowledge graphs since they lead to unpredictable link prediction results. Since we cannot prove consistent impact of redundancies, the lower performance of link prediction on knowledge graphs with redundancy removed cannot be explained by a lack of redundancy. We argue that removing redundancy from knowledge graphs is not a valid method of handling redundancy, and should not be used as a solution to the problem that redundancies present.

Acknowledgments

I would like to thank my thesis advisor, Prof. Carlos Rivero, for providing advice and encouragement to guide me through the process of writing this thesis. Thank you for introducing me to research and providing me with opportunities to develop my skills as a researcher. I am also grateful to the other members of my committee, Prof. Zachary Butler and Prof. Matthew Fluet, for the time they dedicated to understanding this work, asking thoughtful questions, and providing helpful feedback. Thank you to everyone in and adjacent to the theory lab for answering my questions about being a graduate student and celebrating all my milestones along the way. Thank you to my roommates, friends, and family for lending an ear when needed, and supporting me always.

To my parents, for always believing in me.

Contents

1 Introduction **1**

2 Background **5**
2.1 Knowledge Graphs 5
2.2 Benchmarking Knowledge Graphs 5
2.3 Redundancy in Knowledge Graphs 6
2.4 Cardinality 7
2.5 Measuring Redundancy: Anomaly Coefficient 8
2.6 Quantifying Rules: Support and Confidence 9
2.7 Link Prediction 10

3 Related Work **14**

4 Redundancy Measurements **18**
4.1 Measuring Redundancy with Rules 18
4.2 Predicate-Specific Redundancy Metric $\delta(p)$ 19
4.3 Link Prediction Evaluation Adjustment 19
4.4 Overall Redundancy Measurement 20

5 Cardinality Measurements **22**
5.1 Measuring Cardinality with Rules 22

6 Implementation **24**
6.1 Rule Implementation 24

7 Results **28**
7.1 Original vs. Redundancy-Removed Knowledge Graphs 28
7.2 Cartesian Product 31
7.3 Cardinality 33
7.4 Overall Redundancy Measurement 34
7.5 Adjusting Link Prediction Evaluation with $\delta(p)$ 36

8 Conclusion **45**

Appendices **52**

A Implementation Appendix **53**
A.1 Cypher Queries 53
A.1.1 Cartesian product redundancies with Cypher 53

List of Figures

7.1 Distribution of different types of redundancies across different knowledge graph splits in FB15k. 29
7.2 Distribution of different types of redundancies across different knowledge graph splits in FB15k-237. 30
7.3 Distribution of different types of redundancies across different knowledge graph splits in WN18. 31
7.4 Distribution of different types of redundancies across different knowledge graph splits in WN18RR. 32
7.5 Overall redundancy value for the training split of each knowledge graph, grouped by how much each type of redundancy contributes to the final measurement. 36
7.6 Overall redundancy value for the test split of each knowledge graph, grouped by how much of each type of redundancy is present in the test split. 37
7.7 Unweighted link prediction results with the RR metric. 38
7.8 Gain of weighted link prediction results with the RR metric. . . 41
7.9 Gain of weighted link prediction results with the BPM@10 metric. 43
7.10 Gain of weighted link prediction results with the SetR metric. . 44

A.1 Counting the number of triples with the predicate `_verb_group` 53
A.2 Measuring the support of the Cartesian product rule 54
A.3 Measuring the denominator of confidence for Cartesian product 54

List of Tables

4.1 Rules and metrics to quantify each type of redundancy. 18

5.1 Rules and metrics to quantify cardinality. 22

7.1 Overall redundancy value for each knowledge graph. 35
7.2 Unweighted link prediction results. 39
7.3 Link prediction results weighted with predicate-specific redundancy measurements. 42

Chapter 1

Introduction

Knowledge graphs represent real-world data in a directed graph format where two entities, connected by a predicate, represent one fact [13]. Link prediction, which is the prediction of new relationships using existing entities and predicates, is currently a major area of research related to knowledge graphs. Link prediction is done using embedding models, which predict missing links by ranking possible new facts from most to least likely [6]. Link prediction is typically evaluated with mean reciprocal rank (MRR) and hits@k [7,24]. This research relies on the information retrieval versions of these metrics: reciprocal rank (RR) and recall@k ($R@k$), which are equivalent, in our implementation, to MRR and hits@k. We also use a discount cumulative gain metric $BPM@k$ which is intended to measure relevance among the top k results [28]. Link prediction models use benchmarking knowledge graphs for training and testing [19]. Throughout this thesis, benchmarking knowledge graphs (which are specific, commonly used instances of larger knowledge graphs) will be referred to as knowledge graphs. The knowledge graphs analyzed in this thesis are: FB13 [20], FB15k [6], FB15k-237 [24], WN11 [5], WN18 [6], WN18RR [7], BioKG [26], Hetionet [12], NELL-995 [27], and YAGO3-10 [15]. These are ten of the most frequently used benchmarking knowledge graphs for link prediction.

Problem Statement Redundancies exist within benchmarking knowledge graphs that allegedly artificially improve the results of link prediction [24]. For example, WN18 contains symmetric redundancies, as seen in the predicate

`similar to` and demonstrated by the facts `(happy, similar to, joyful)` and `(joyful, similar to, happy)`. There are also redundancies that occur across multiple different predicates, such as near-reverse redundancy. This occurs with the WN18 predicates `has part` and `part of`, demonstrated by the facts `(sunrise, has part, sun)` and `(sun, part of, sunrise)` [7].

This thesis studies five types of redundancy that are commonly discussed in the link prediction field: near-reverse, near-duplicate, symmetric, transitive, and Cartesian product. Studying these five types of redundancy across ten knowledge graphs will provide new insights into the composition of the information in each knowledge graph. This is important to know because of the potential effects on link prediction.

It is implied by related work that redundancies have a negative impact on link prediction. Toutanova and Chen [24] showed that a simple observed feature model could perform better than state-of-the-art embedding models on FB15k, a knowledge graph they found to have 80.9% near-reverse or near-duplicate triples. This implies that if a model scores well on a highly redundant knowledge graph, it may not directly correlate to good performance on other more complex knowledge graphs that do not share the same level of redundancy. One hypothesis of this thesis is:

> The presence of redundancies in benchmarking knowledge graphs makes link prediction less challenging, which results in better scores on link prediction evaluation metrics.

To attempt to solve this issue, knowledge graphs with some redundancies removed have been created. For example, Toutanova and Chen [24] created FB15k-237 by removing predicates with a redundancy level that crossed a certain threshold. Another hypothesis of this thesis is:

> Removing facts to create a new benchmarking knowledge graph without redundancies is a flawed method because it removes true facts that belong in the graph, which leads to unrealistic link prediction evaluation.

Using the example above, if the redundant fact `(joyful, similar to, happy)` is removed, the word `joyful` will not be considered similar to `happy`, even though it should be. We investigate how legitimate removing redundancy is as technique for handling highly redundant knowledge graphs.

Contributions This research establishes two main goals: measure redundancies in benchmarking knowledge graphs and adjust the measurement of link prediction evaluation based on levels of redundancy. The completion of these goals allows the analysis of the impact of redundancies on link prediction.

In order to measure redundancy, different types of redundancies must be defined and quantified. We use Horn rules to define five types of redundancies: near-duplicate, near-reverse, symmetric, transitive, and Cartesian product. The rules are quantified using the head coverage and confidence metrics. This is the first time these five types of redundancies have been measured in a consistent method.

The metric $\delta(p)$ is defined to be the aggregation of the values for all different types of redundancies for a predicate p. Tiwari et al. [23] previously created an "anomaly coefficient" to measure near-duplicate, near-reverse, and Cartesian product redundancies in knowledge graphs. However this technique only allowed the maximum of those three types of redundancies to be represented. In knowledge graphs like FB15k, all five types of redundancies can be found at high levels for one predicate. $\delta(p)$ allows all redundancies to be represented simultaneously for a more full representation of the data.

Since each benchmarking knowledge graph analyzed in this thesis contains several predicates– anywhere from 11 to 1345– we define another metric to aggregate the predicate-specific $\delta(p)$ metric into one value. This is called the overall redundancy measurement. Knowledge graphs that are known to be highly redundant, such as WN18 and FB15k, have high values for the overall redundancy measurement which confirms its ability to accurately represent redundancy. The knowledge graph BioKG has a large overall redundancy measurement due to a high amount of transitive redundancy, which has not been previously studied.

Adjusting link prediction evaluation is done by using the redundancy value of each individual predicate, $\delta(p)$, as a weight to calculate the weighted mean of each query asked in link prediction.

Results All knowledge graphs analyzed in this thesis have some level of redundancy, however after weighting link prediction metrics based on this redundancy, some showed decreased performance and some showed increased performance. Decreased values varied in significance. These mixed results refute the first hypothesis of this thesis, which goes against the commonly-

believed theory that redundancies have a consistent impact on link prediction. As such, our second hypothesis is proven to be true: removing redundancies is not a legitimate method for handling redundancies.

Taking redundancies into account should be an essential step for anyone using the benchmarking knowledge graphs analyzed in this thesis. This is especially important considering that the impact of redundancies on link prediction is not consistent. To the best of our knowledge, this is the first method for offsetting redundancies in the knowledge graph field and it provides new insights for the effects of redundancy on link prediction.

Organization The organization of this thesis is as follows. Chapter 2 introduces the necessary background for understanding knowledge graphs, link prediction, association rules, and more. Chapter 3 discusses relevant related work, including which redundancies have already been studied, and which have only been introduced theoretically. Chapter 4 contains the methods we created for measuring redundancy, including $\delta(p)$, the overall redundancy measurement, and methods for adjusting link prediction results. Chapter 5 contains the methods we created for measuring cardinality-related properties in knowledge graphs. We introduce these methods, but did not pursue further experiments, choosing instead to focus on the five main redundancies defined in Chapter 4. Chapter 6 discusses the implementation of all of the methods discussed in the previous two chapters, as well as providing a link to the repository where these are found: `https://github.com/lek7518/AnomalyResearch`. Chapter 7 contains the results of this thesis. First, the redundancy types described in Chapter 4 are measured using methods defined by Tiwari et al. [23] to aggregate values. Then, a short discussion of the limited cardinality experiments is included. Section 7.4 is where results of the overall redundancy measurement are discussed, including the final measurement for each knowledge graph, as well as a discussion of redundancies in training versus test splits. Section 7.5 contains results of the link prediction evaluation adjustment and discussions about the impact of redundancies on link prediction. Chapter 8 concludes the thesis, with final thoughts and future work. After the Bibliography, there is an Appendix with additional information pertaining to Cypher implementations of rule-based redundancy measurements.

Chapter 2

Background

2.1 Knowledge Graphs

A knowledge graph is a graph containing data that represents real-world facts [13]. Each fact is stored as a "triple" composed of three parts: subject, predicate, and object, represented as `(s, p, o)`. The subject and object are entities and the predicate is the relation that connects them. For example, the triple `(Greta Gerwig, director, Barbie)` represents the fact that Greta Gerwig was the director of the movie Barbie. Knowledge graphs are used by many different industries, including artificial intelligence, social media, advertising, healthcare, information retrieval, and web searching [2, 17].

2.2 Benchmarking Knowledge Graphs

Several benchmarking knowledge graphs were created in order to train link prediction models and test their efficacy. The knowledge base WordNet contains lexical information about words and the relations between them. Some of the predicates include `_similar_to`, a symmetric predicate, and `_hypernym` and `_hyponym`, a set of near-reverse predicates. An example WordNet triple is `(_average_2, _similar_to, _moderate_8)`, denoting that the word "average" is similar to the word "moderate". Three notable benchmarking knowledge graphs have been created from WordNet: WN11 [5], WN18 [6], and WN18RR [7]. Other benchmarking knowledge graphs are derived from FreeBase, a knowledge base with data across many different topics. Notable bench-

marking knowledge graphs include FB13 [20], FB15k [6], and FB15k-237 [24] which contain information pertaining to people, organizations (such as governments and educational institutions), and popular culture (such as sports, films, and music). Other commonly used benchmarking knowledge graphs include BioKG [26] and HetioNet [12] which contain healthcare-related data, and NELL-995 [27] and YAGO3-10 [15] which contain more general information.

2.3 Redundancy in Knowledge Graphs

Toutanova and Chen discovered that there was so much redundancy in FB15k that a simple model based on observed features could outperform the top latent models at the time [24]. The features they found will be referred to as near-duplicate and near-reverse redundancies. A *near-duplicate* redundancy is a pair of triples that have the same subject and object, but different predicates. For example, `(Greta Gerwig, director, Barbie)` and `(Greta Gerwig, writer, Barbie)` show the near-duplicate relationship between the predicates `director` and `writer`. A *near-reverse* redundancy is a pair of triples that have inverse subjects and objects, and different predicates. For example, the triples `(Barbie, has actor, Margot Robbie)` and `(Margot Robbie, known for, Barbie)`. Link prediction on these types of redundancy is very easy for a model to learn, but the information gained by the prediction is trivial.

Toutanova and Chen [24] removed redundancies from FB15k to create FB15k-237, a knowledge graph intended to be more challenging and realistic for link prediction. Dettmers et al. [7] used the same methods on WN18 to create WN18RR. These knowledge graphs were considered by some as the new benchmark for link prediction models. This thesis will discuss how successful these attempts at removing redundancy were.

More patterns within benchmarking knowledge graphs have been recognized and named by other contributors. Rossi et al. identified further properties of knowledge graphs, including symmetric, reflexive, and transitive relations [19]. *Symmetric* redundancies are represented by pairs of triples that have inverse subjects and objects, and the same predicate. For example, the predicate `spouse` is symmetric:

`(Greta Gerwig, spouse, Noah Baumbach),`
`(Noah Baumbach, spouse, Greta Gerwig).`

Reflexive redundancies occur when a predicate connects an entity to itself. *Transitive* redundancies follow the transitive property with three triples that have the same predicate. For example, the triples

```
(NYC, located in, NY),
(NY, located in, USA),
(NYC, located in, USA)
```

form a transitive relationship. Akrami et al. [2] identified *Cartesian product* redundancies. These occur when all subjects and all objects of a certain predicate are all connected to one another. An example of this is the predicate `works with` which represents colleagues. In a knowledge graph representing actors and directors working on a specific movie, every director and actor under this relation are connected, because each one `works with` the other.

This paper will analyze ten benchmarking knowledge graphs for the aforementioned patterns.

2.4 Cardinality

Cardinality of data has also been assessed as something that impacts link prediction. Bordes et al. [6] define four distinct classes to represent cardinality: 1-to-1, 1-to-Many, Many-to-1, and Many-to-Many. These classes represent the relationship from heads (subjects) to tails (objects) in triples. For a relation with 1-to-1 cardinality, one head corresponds to one tail; for 1-to-Many, one head may have several tails; for Many-to-1, several heads may have the same tail; for Many-to-Many, each head has several tails and each tail has several heads. Bordes et al. [6] classify relations by taking the average of how many tails each head has, and the average of how many heads each tail has. These numbers give the first and second value respectively for the cardinality. The value shall be 1 if the average is below 1.5, and Many otherwise.

The impact of these cardinalities on link prediction has not been thoroughly evaluated. Bordes et al. [6] found that it is easiest to predict the head of 1-to-Many relations and the tail of Many-to-1 relations. Akrami et al. [2] used these cardinalities to classify FB15k-237 and WN18RR and found similar results: models scored much better on the "1" side of 1-to-Many and Many-to-1 relations. Rossi et al. [19] made similar observations, refering to this phenomenon as source and target peers. The impact of other cardinalities on link prediction have not been confirmed, as it is difficult to see the impact in

isolation when many knowledge graphs have a variety of other redundancies already affecting link prediction.

2.5 Measuring Redundancy: Anomaly Coefficient

Methods of measuring some of the aforementioned types of redundancies in data have been created by Tiwari et al. [23]. They refer to redundancies as anomalies, and created "anomaly coefficients" to quantify near-duplicate, near-reverse, and Cartesian product properties in data.

Given a whole graph G and a predicate p, the anomaly coefficient to measure the Cartesian product redundancy of p is:

$$\mathrm{cp}(G,p) = \frac{|T(G,p)|}{|S(G,p)|\ |O(G,p)|} \tag{2.1}$$

where $T(G,p)$ represents triples in G with predicate p, $S(G,p)$ represents subjects, and $O(G,p)$ represents objects. A score of $\mathrm{cp}(G,p) = 1$ would indicate that all subjects are connected to all objects for the given predicate p.

The anomaly coefficient to measure the near-same (near-duplicate) redundancy of p is:

$$\mathrm{ns}(G,p) = \sup_{p'} \frac{|R(G,p) \cap R(G,p')|}{|T(G,p)|} \tag{2.2}$$

where $R(G,p) = \{(s,o)|(s,p,o) \in G\}$, or in other words, the set of subject-object pairs with predicate p. Taking the supremum with p' finds the p' that gives the largest set of common subject-object pairs between p and p'. This ensures the near-same measurement for p will represent the greatest possible redundancy value. If $R(G,p) = R(G,p')$, meaning p and p' have all the same subject-object pairs, then $\mathrm{ns}(G,p) = 1$, indicating the highest possible redundancy.

The anomaly coefficient to measure the near-reverse redundancy of p is:

$$\mathrm{nr}(G,p) = \sup_{p'} \frac{\max(|R(G,p) \cap R^{-1}(G,p')|, |R^{-1}(G,p) \cap R(G,p')|)}{|T(G,p)|} \tag{2.3}$$

where $R^{-1}(G,p) = \{(o,s)|(s,p,o) \in G\}$, or inverse pairs with predicate p. This measurement compares subject-object pairs in p with inverse pairs in

p', and vice versa. Taking the maximum of these values allows the greatest possible redundancy value to be represented. If $R(G,p) = R^{-1}(G,p')$ or $R^{-1}(G,p) = R(G,p')$, meaning every pair in p and p' are inverse of each other, then $\mathrm{nr}(G,p) = 1$.

2.6 Quantifying Rules: Support and Confidence

One of the goals of this thesis is to identify several types redundancies in benchmarking knowledge graphs in a more consistent way. Redundancies are represented as association rules and there are several ways they can be quantified. Rules will be represented as Horn rules: $\vec{B} \Rightarrow (s,p,o)$ where $\vec{B}$ is the body of the rule and (s,p,o) is the head of the rule. $\vec{B}$ may contain one or more triples with s, p, or o values that may differ from the head of the rule. The **support** of a rule is the number of distinct subject-object pairs in the head that make the whole rule, $\vec{B}$ and (s,p,o), true. This is defined below as

$$\mathrm{supp}(\vec{B} \Rightarrow (s,p,o)) = |\{(s,o)|\vec{B} \wedge (s,p,o) \in G\}| \tag{2.4}$$

where G is the knowledge graph. Both $\vec{B}$ and (s,p,o) for a specific s and o must occur in G in order for that (s,o) pair to count towards the support of the rule.

Galárraga et al. [11] define **head coverage** as support in the context of the size of the predicate. This is calculated by dividing the support by the number of triples with the current predicate.

$$\mathrm{hc}(\vec{B} \Rightarrow (s,p,o)) = \frac{\mathrm{supp}(\vec{B} \Rightarrow (s,p,o))}{|T(G,p)|} \tag{2.5}$$

This represents the support in a more accessible way. Near-duplicate, near-reverse, transitive, symmetric, and cardinality-related relations will be quantified using head support.

Galárraga et al. [11] also define **standard confidence** as support divided by the number of distinct subject-object pairs that make the body of the rule true:

$$\mathrm{conf}(\vec{B} \Rightarrow (s,p,o)) = \frac{\mathrm{supp}(\vec{B} \Rightarrow (s,p,o))}{|\{(s,o)|\vec{B} \in G\}|} = \frac{|\{(s,o)|\vec{B} \wedge (s,p,o) \in G\}|}{|\{(s,o)|\vec{B} \in G\}|} \tag{2.6}$$

When calculating the denominator, the head of the rule with (s, o) as the subject and object may be true or false; i.e. the specific triple (s, p, o) may not exist. The confidence measurement gives the ratio of how many times the head is true. Cartesian product redundancies are quantified using confidence.

The standard confidence takes a closed world approach, meaning that there is an assumption that anything not represented in the knowledge graph is false [13]. In some cases, a closed world assumption is useful. For example, `(Quentin Tarantino, director, Barbie)` would not be in our example knowledge graph because it is not true. However, the goal of link prediction is to find facts not currently in the knowledge graph that are true. Because of this, an open world assumption is required. Another way to quantify confidence with an open world assumption is partial completeness assumption (PCA) confidence, which allows only the subject (object) to have an open world assumption, and the object (subject) has a closed world assumption [11]. Instead of standard confidence, PCA confidence can be calculated as an alternative to the closed world assumption. In the case of Cartesian product redundancies, PCA confidence is equivalent to standard confidence due to the composition of the rule. We continue to use standard confidence for simplicity.

2.7 Link Prediction

A key feature of knowledge graphs is link prediction: the task of predicting relationships that are not yet recorded between entities in the knowledge graph. This is intended to fill in missing facts in incomplete knowledge graphs. For example, suppose the knowledge graph containing the triple `(Greta Gerwig, director, Barbie)` did not have any information about the writer of the Barbie movie. Link prediction may be able to predict the triple `(Greta Gerwig, writer, Barbie)` based on the existing information about writers and directors in the knowledge graph.

Many models with different methods for link prediction exist. We used several link prediction models that are publicly available, and were trained for different tasks related to link prediction [14,18]. A model was trained for each link prediction method and knowledge graph. The link prediction methods are as follows: BoxE [1], ComplEx [25], HAKE [30], HolE [16], QuatE [29], RotatE [22], RotPro [21], TorusE [9], TransE [6] and TuckER [4]. These methods are representative in the sense that they all use different approaches to accom-

plish link prediction. BoxE represents entities, predicates and triples as rectangles in the Euclidean space; ComplEx, RotatE and RotPro use different operations over complex numbers; QuatE employs quaternion numbers; HAKE, HolE and TransE represent entities and predicates in the Euclidean space and combine them using circular correlations and L1/L2 distances. TorusE represents them in a torus space. TuckER represents them as tensors and applies the Tucker decomposition to combine them.

Redundancies in knowledge graphs are hypothesized to have an impact on the results of link prediction [7, 24]. Link prediction is used to predict the subject or object of a triple. A question of the form $q(s, p, ?)$ is used to predict the object for one triple, and $q(?, p, o)$ is used to predict the subject. Link prediction models then predict the missing entity, ?, by ranking all of the possible entities based on which would be the best fit for that triple. Rank 1 is the best possible rank. Predicting the object of the triple `(Greta Gerwig, writer, Barbie)` is represented by `(Greta Gerwig, writer, ?)`. An accurate prediction model would rank the entity `Barbie` as number 1, and then other likely movies in descending order after that.

When training link prediction models, a known triple is used to test the ability of the model. For each triple in a testing set G_{Test}, the questions for subject and object are asked. The subject or object that is original to that triple is considered the correct or *positive* response. The goal for every model is to return the positive entity as rank 1. Any subject or object returned that creates a triple that already exists in G is considered a false positive, and may be filtered out of the ranked results as described below. The rank that the positive achieves for each question asked is considered the result of link prediction evaluation.

Filtering methods exist to avoid penalizing models for predictions that are semantically correct. If a model ranks another triple that exists in G higher than the target triple, it should not be penalized for making a correct prediction. If a model was predicting `(Greta Gerwig, director, ?)` and the top ranked answer was `Ladybird` but the the positive entity was `Barbie`, the model would not score a rank of 1. In this example, both are correct facts that exist in G, so some may argue that the model should still be able to score a rank of 1. Filtering removes the entities that form triples that exist in the knowledge graph from the list of predictions before they are ranked. In the example above, `Ladybird` would be removed from the ranks, along with

any other entity, `o`, where a triple `(Greta Gerwig, director, o)` exists in G. After these false positives have been removed, the rest of the entities can be ranked.

The entities that remain after filtering are *negatives*. The number of negatives is represented as $|N|$. Every entity other than the positive entity is now confirmed to be a negative example of something the model should not rank highly (assuming a closed world approach). The maximum filtered rank is equal to the number of negatives.

The method of ranking results for relevance is borrowed from the information retrieval field. For example, if you used a search engine for a query, you would want the top 10 most relevant web pages to appear first. Link prediction evaluation borrows several metrics from the information retrieval realm in order to aggregate rank data for use in analyzing the effectiveness of a model. The simplest metric is mean rank (MR), or arithmetic mean rank (AMR), which is the average of all the rank values. This is defined as:

$$\text{MR} = \frac{1}{2|G|} \sum (r_s + r_o) \tag{2.7}$$

where G represents all of the triples in the knowledge graph being queried, and r_s and r_o represent the rank of the positive triple in $q(s, p, ?)$ and $q(?, p, o)$ respectively.

Another commonly used method for aggregating ranks is mean reciprocal rank (MRR). This is the reciprocal of MR, which constrains the possible values between 0 and 1.

$$\text{MRR} = \frac{1}{2|G|} \sum (\frac{1}{r_s} + \frac{1}{r_o}) \tag{2.8}$$

The best possible value is $\frac{1}{1}$, and the worst possible value is $\frac{1}{|G|}$, which would be close to zero. There is controversy surrounding the accuracy of MRR as a metric, due to values not scaling linearly [10]. Since it is so widely regarded in knowledge graph and link prediction assessment, it will be included along with other more accurate metrics for context.

Standard information retrieval methods use reciprocal rank (RR) to represent the mean reciprocal rank, since taking the mean of all results is implied. In results, MRR will be represented by RR.

Filtered mean rank (FMR) is the average rank after filtering has been completed.

$$\text{FMR} = \frac{1}{|N_s| + |N_o|} \sum (r_{f(s)} + r_{f(o)}) \tag{2.9}$$

where $r_{f(s)}$ is the filtered rank and $|N_s|$ is the number of negatives when predicting the object. Similarly, $r_{f(o)}$ is the filtered rank and $|N_o|$ is the number of negatives when predicting the subject.

Filtered mean reciprocal rank (FMRR) is the reciprocal of FMR.

$$\text{FMRR} = \frac{1}{|N_s| + |N_o|} \sum (\frac{1}{r_{f(s)}} + \frac{1}{r_{f(o)}}) \tag{2.10}$$

Ranks r_s and r_o in the following equations may be filtered or not. Ranks in the experiments of this paper will all be filtered.

Another common metric used to measure link prediction performance is hits@k. This defines how many times the positive entity was ranked in the top k results.

$$\text{Hits@}k = \frac{1}{2|G|} \sum \begin{cases} 0 \text{ if } r \leq k \\ 1 \text{ otherwise} \end{cases} \tag{2.11}$$

The standard information retrieval metric recall@k, or $R@k$, also represents this information. $R@k$ is represented as the total number of relevant items in the top k ranks divided by the total number of relevant items. In this context, all negatives are considered the relevant items, so $R@k$ is used in results as another representation of hits@k.

The information retrieval metric $BPM@k$ is also used [28]. $BPM@k$ is a discount cumulative gain function, which measures the amount of relevance seen while traversing documents, or in this case the first k-ranked triples. The $BPM@k$ implementation used is as follows

$$BPM@k = \sum_{i=1}^{\tau} \frac{r(e_i)}{(i+1)^2} \quad \text{if } r \leq k \tag{2.12}$$

where the stopping criteria $\tau = k$. This measures the rank of the positive triple within the first k ranks.

Chapter 3

Related Work

Tiwari et al. [23] measure redundancies with "anomaly coefficients". Metrics are introduced to measure Cartesian product, near-same (near-duplicate), and near-reverse predicates. These metrics, shown in Section 2.5, are combined into the anomaly coefficient by taking the maximum value of the three metrics. Each predicate's anomaly coefficient is calculated over the whole knowlege graph G. Predicates are then split into quartiles based on the anomaly coefficient, and performance on predicates in each quartile is reported (using $\overline{WMR}$) for several different models. They find that predicates in the highest anomaly coefficient quartile, Q_1, are easier to predict with AMIE. But other models do not follow this hypothesis.

Galárraga et al. [11] develop the rule mining model AMIE, inspired by association rule mining. AMIE functions with an open world assumption, allowed by the partial completeness assumption (PCA). The authors define several metrics for measuring support, head coverage, standard confidence, and PCA confidence (Section 2.6). They also define functionality and inverse functionality. For functional predicates, each head has at most one tail (i.e. `(Margot Robbie, birthday, July 2)`). For inverse predicates, each tail has at most one head. It should be noted that this is different than head (tail) cardinality, which puts emphasis on the extent of the commonality of the tail (head), and not the uniqueness of the head (tail).

Bordes et al. [6] created the TransE model, which uses embeddings to represent entities and models relationships between entities as translations. These authors introduced the idea of using filtered metrics when analyzing model

performance. They also define cardinality in four classes: 1-to-1, 1-to-Many, Many-to-1, and Many-to-Many. These classes are computed by computing h-to-t, where h is the average number of heads per triple in a predicate, and t is the average number of tails per triple, as described in Section 2.4. The threshold between 1 and Many is 1.5. Using the TransE model on the FB15k knowledge graph with predicates divided between the cardinality classes, they observed predicting entities on the 1 side of triples (commonly connected entities) was easier (higher hits@10 values).

Toutanova and Chen [24] displayed the influence of redundancies in knowledge graphs by comparing the efficacy of a latent (embedded) model and an observed feature model. The observed features included near-duplicate and near-reverse predicates, as well as frequently appearing entities (such as `United States` as a common entity for the predicate `nationality`). The observed feature models that used all of these features had better MRR and hits@10 results than all latent models (such as DistMult and TransE). Redundancies were measured in the training set. The authors report the percent of test set triples with a duplicate or reverse triple in the training set is 80.9% for FB15k and 94.0% for WN18. They created the FB15k-237 (FB15kSelected) knowledge graph by removing duplicate and reverse relations with 97% redundancy or more. Latent models scored better on FB15k-237 than observed feature models, since redundancies were removed.

Dettmers et al. [7] use an inverse model: a rule-based model that uses only inverse (near-reverse) predicates to predict missing entities. The threshold for a predicate to be considered inverse is $0.99 - (f_v + f_t)$, where f_* represents the fraction of triples in the validation and test sets, since these triples are not analyzed. The inverse model achieves state-of-the-art results on knowledge graphs WN18 and FB15k, but does not do well on FB15k-237. The authors use the same methods as [24] to create WN18RR, a version of WN18 with near-reverse predicates removed. The inverse model scores only slightly lower than other models because symmetric relations, such as `similar to`, were not removed. The authors created ConvE, a convolutional model that works well on knowledge graphs with a high indegree (very connected or complex knowledge graphs like YAGO-10 and FB15K-237, but not so much WN18). WN18RR's symmetric predicates and low indegree value means ConvE didn't work as well on it.

Akrami et al. [2] completed an analysis on near-duplicate, near-reverse,

and Cartesian product triples that shows the impact they have on link prediction. They found these redundancies led to a substantial over-estimation of model accuracy. They claim, similarly to [7], that WN18 and FB15k should no longer be used. They created YAGO3-10-DR by removing near-duplicate and symmetric relationships from YAGO3-10. They also analyzed which embedding models performed best on each of the cardinality classes defined by [6]. They confirmed all methods performed better when predicting the "1" side of 1-to-Many or Many-to-1 relations.

Akrami et al. [2] define these redundancies in measurable ways. They defined near-duplicate as the following:

$$\frac{|T_{r_1} \cap T_{r_2}|}{|r_1|} > \theta_1 \text{ and } \frac{|T_{r_1} \cap T_{r_2}|}{|r_2|} > \theta_2 \tag{3.1}$$

where $|r|$ is the number of triples in the relation r (i.e. the predicate p), T_r represents (s, p, o) triples in the predicate p or relation r, and θ represents the threshold. For FB15k, $\theta = 0.8$.

Reverse duplicate (near-reverse) pairs are defined as:

$$\frac{|T_{r_1} \cap T_{r_2}^{-1}|}{|r_1|} > \theta_1 \text{ and } \frac{|T_{r_1} \cap T_{r_2}^{-1}|}{|r_2|} > \theta_2 \tag{3.2}$$

where T_r^{-1} represents inverse triples (o, p, s).

Cartesian product is defined as:

$$\frac{|r|}{(|S_r| \times |O_r|)} > \theta \tag{3.3}$$

where $|S_r|$ and $|O_r|$ are the set of all subjects and objects with relation r respectively.

Rossi et al. [19] do comparative analysis with many models and knowledge graphs. They define several relational properties that can be considered redundancies in a rule-based way, including: reflexivity, irreflexivity, symmetry, anti-symmetry, and transitivity. They also define structural features such as number of peers, degree of the original reified relation, and more. Peers are defined as head peers ($h' \in E|(h', r, t) \in G_{train}$) and tail peers ($t' \in E|(h, r, t') \in G_{train}$). Tail peers are the known alternatives to t when the head is h and the relation is r. Source and target peers are defined as the

source (known entity) and target (entity being predicted) entity of a prediction. For head prediction, source peers are tail peers and target peers are head peers. For tail prediction, the reverse is true. They find a greater number of source peers leads to better hits@1 results, and a greater number of target peers leads to worse hits@1 results. They also analyze the impact of original reified relation on databases where new relationships have been created to represent multi-degree relationships, such as FreeBase. They find that higher degrees correspond to better prediction results, because of the richness of the corresponding cliques in the knowledge graph.

Recent related work includes the exploration of other types of redundancies, such as the Z-paradox [8].

Chapter 4

Redundancy Measurements

4.1 Measuring Redundancy with Rules

In order to have clear definitions of different types of redundancies, we represent them as association rules. This allows the quantification of instances of the redundancies as the head coverage or the confidence of the rule. For consistency, the head of a rule is always represented by the triple `(s, p, o)`. Any variable `s`, `p`, or `o` is strictly different from their counterpart `s'`, `p'`, or `o'`.

Near-duplicate and near-reverse redundancies are quantified with the supremum of `p'` of the head coverage of the rule. Transitive and symmetric redundancies are quantified using head coverage. Cartesian product redundancies

Name	**Rule** (R)	**Metric**
Near-Duplicate	`(s, p', o)` $\Rightarrow$ `(s, p, o)`	$\sup_{p'}(\mathrm{hc}(R))$
Near-Reverse	`(o, p', s)` $\Rightarrow$ `(s, p, o)`	$\sup_{p'}(\mathrm{hc}(R))$
Cartesian Product	`(s, p, o')` $\wedge$ `(s', p, o)` $\Rightarrow$ `(s, p, o)`	$\mathrm{conf}(R)$
Transitive	`(s, p, x)` $\wedge$ `(x, p, o)` $\Rightarrow$ `(s, p, o)`	$\mathrm{hc}(R)$
Symmetric	`(o, p, s)` $\Rightarrow$ `(s, p, o)`	$\mathrm{hc}(R)$

Table 4.1: Rules and metrics to quantify each type of redundancy.

are computed with standard confidence (defined in Section 2.6).

4.2 Predicate-Specific Redundancy Metric $\delta(p)$

For each predicate in a knowledge graph, each type of redundancy is quantified using the rules and equations in Table 4.1. For a single predicate, these values are combined using the absolute error, defined as

$$\text{Absolute Error} = \sum_{i=1}^{n} |y_i - x_i| \tag{4.1}$$

where y is the is the predicted value and x is the redundancy value for a type of redundancy i. The value used for y is zero, to represent that no redundancy is expected. This is simplified to represent the combination of all redundancy values for one predicate p as $\delta(p)$,

$$\delta(p) = \sum_{i=1}^{5} x(p)_i \tag{4.2}$$

where the value for each type of redundancy for a predicate p is $x(p)_i$, where i indicates one of the five types of redundancies defined in Table 4.1.

Analysis showed that measurements for all types of redundancies can be high for the same predicate. This means $\delta(p)$ can range from 0 to 5.

4.3 Link Prediction Evaluation Adjustment

The predicate-specific redundancy value, $\delta(p)$, is used as a weight to adjust link prediction results to more accurately represent performance. In order to use $\delta(p)$ to adjust link prediction metrics, the values must be normalized as follows

$$\delta_{normal}(p) = \left(\frac{1}{\delta(p) + 1}\right)^2 \tag{4.3}$$

This is necessary because $\delta(p)$ yields a higher value on a knowledge graph with more redundancy. When adjusting link prediction results, a lower value

is required to allow high redundancy values to offset the artificially high link prediction values that redundancies create.

To adjust each link prediction result, the weighted mean was used.

$$\overline{x}_w = \frac{\sum_{i=1}^{n} w_i x_i}{\sum_{i=1}^{n} w_i} \tag{4.4}$$

For this application, the weight w will be the normalized redundancy value for each predicate, $\delta_{normal}(p)$, and x is the value of the link prediction metric, $M(q(s,p,o))$.

$$\overline{x}_\delta = \frac{\sum \delta(p)_{normal}\ M(q(s,p,o))}{\sum \delta(p)_{normal}} \tag{4.5}$$

$\overline{x}_\delta$ is used to adjust link prediction evaluation. This calculation is done on existing link prediction results calculated across nine knowledge graphs using ten models (as seen in the unweighted results in Table 7.2). The weighted adjustment is done using standard information retrieval methods. The results of link prediction are assessed with the metrics RR, $R@k$, and $BPM@k$ using the `cwl_eval` [3] library. For each link prediction query, $q(s,p,o)$, one of the aforementioned metrics, M, is computed, then the weighted mean ($\overline{x}_\delta$) of each $M(q(s,p,o))$ value is taken using $\delta_{normal}(p)$ as the weight. Results follow in Section 7.5.

4.4 Overall Redundancy Measurement

In order to get a singular redundancy value for an entire knowledge graph, $\delta(p)$ for each predicate p was averaged using the number of triples in each predicate as the weight. This allows a minimum value of 0.0 and a maximum value of 5.0. Using the weighted mean (Equation 4.4) ensures the balance of triples in the knowledge graph are accurately represented. This calculation is represented as

$$\frac{\sum_{p=0}^{n} \delta(p)\ |T(G,p)|}{|G|} \tag{4.6}$$

where n is the number of predicates, $|T(G,p)|$ is the number of triples in the graph G with predicate p, and $|G|$ is the number of all the triples in G. These results are reported in Table 7.1.

In order to isolate the different types of redundancies, this calculation is rearranged to the equivalent equation

$$\sum_{i=1}^{5} \frac{\sum_{p=0}^{n} x(p)_i \; |T(G,p)|}{|G|} \tag{4.7}$$

where $x(p)_i$ is the redundancy value for predicate p with redundancy type i. This allows the creation of Figures 7.5 and 7.6.

Chapter 5

Cardinality Measurements

5.1 Measuring Cardinality with Rules

The two values from the Head and Tail Cardinality rules create the cardinality classifications seen in Section 2.4. If both values are high, this is comparable to a Many-to-Many classification. If the head cardinality value is low and the tail cardinality value is high, it is similar to 1-to-Many; if the head value is high and the tail value is low, it is similar to Many-to-1; and if both values are low, then it is similar to 1-to-1. This classification will allow analysis of the redundancy in comparison to traditional cardinality labels.

Calculating cardinality using the head coverage of two rules without thresholds provides a different perspective than previous research. Bordes et al. [6] computed their threshold for cardinality by taking the average of heads (subjects) per tail (objects) for a predicate (Section 2.4). They chose 1.5 as their threshold between "1" and "Many". Using the head coverage method, there will be support for a tail if there are at least two tails per head. If we assume every head either has one tail or two tails, the Bordes 1.5 threshold equates

Name	**Rule** (R)	**Metric**
Head Cardinality (N:1)	`(s', p, o)` $\Rightarrow$ `(s, p, o)`	hc(R)
Tail Cardinality (1:N)	`(s, p, o')` $\Rightarrow$ `(s, p, o)`	hc(R)

Table 5.1: Rules and metrics to quantify cardinality.

to a head coverage of 0.50 (0.5 * 1 tail + 0.5 * 2 tails = 1.5). However, it is possible for heads to have more than 2 tails. For example, a relation with 60% of heads having only one tail, and 40% of heads having 3 tails, the average comes to 2.2. Bordes et al. would classify this relation as "Many". However the head coverage of the Head Cardinality rule would be 0.4. For this reason, we measure cardinality without using arbitrary thresholds to differentiate between "1" and "Many". The measurements show a spectrum of possible values that cannot be represented with binary labels.

Chapter 6

Implementation

6.1 Rule Implementation

The implementation of the rule-based redundancy measurements was completed using Java. The benchmarking knowledge graphs being analyzed were provided as text files. The Java implementation parses the text files and stores each triple in a Triple object, containing subject, predicate, and object fields. Each Triple is then stored in a dictionary, in which the key is the predicate. This makes analyzing predicate-specific redundancies faster. The support, head support, or confidence is calculated for each type of redundancy using this dictionary. The following implementations can be found at:
`https://github.com/lek7518/AnomalyResearch`

Near-Duplicate These redundancies are measured by comparing each triple with predicate `p` to each triple in every other predicate `p'`. The triples in `p` and `p'` are sorted by `s` value, then `o` value, allowing the comparison to be made with one pass, comparing the lists with pointers (see Algorithm 1). If a match is found (`s = s', o = o'`, using a custom comparator), the support is incremented. After all possible `p'` values have been tried, the supremum ($\sup_{p'}$), or maximum support, is used, as this is the `p'` that has the greatest near-duplicate association with `p`. The maximum support is divided by the size of the list of triples in `p` to find head coverage.

Algorithm 1 Double-pointer search on two sorted lists l_1 and l_2

```
cnt ← 0
p ← 0
q ← 0
while p < len(l1) & q < len(l2) do
    c ← compare(l1[p], l2[q])
    if c ≡ 0 then
        cnt ← cnt + 1
        p ← p + 1
        q ← q + 1
    else if c > 0 then
        q ← q + 1
    else
        p ← p + 1
    end if
end while
```

Near-Reverse A similar method to Near-Duplicate is used for near-reverse redundancies. In order to be able to make one pass through the list, the triples in `p` must be sorted by `s` value first, and the triples in `p'` must be sorted by `o` value first. This allows a similar algorithm to Near-Duplicate to be used, with a different compare function.

Symmetric A similar method to Near-Reverse is used for symmetric redundancies. Two copies of the list of triples in `p` are made: one sorted by `s` values and one sorted by `o` values. Then the same algorithm from Near-Duplicate and Near-Reverse can be used, however no supremum needs to be computed since there is only one value computed for each `p`.

Transitive For each triple with predicate `p`, values are stored in a dictionary with `s` as the key and a list of `o` values as the value. Another dictionary is created to keep track of which `(s, p, o)` values have been seen already, as this identifies the head of the rule, which should only be counted once. As a reminder, the rule is `(s, p, x)` $\wedge$ `(x, p, o)` $\Rightarrow$ `(s, p, o)`. The algorithm looks for each piece of this rule one at a time. As we iterate through each triple,

that triple will be considered the head, `(s, p, o)`. The `s` value is checked against the first dictionary to find another possible `o` value to fulfill `(s, p, x)`. Once this `x` value is found, it is used as a key in the same dictionary. If the original `o` value is found among the list of `o` values under that key, then we have found `(x, p, o)`. If all pieces of the rule are found, the support is incremented and the head is stored in the dictionary to keep track of unique heads. The head coverage is found by dividing the support value for each `p` by the number of triples in `p`.

Cartesian Product For each triple with predicate `p`, the confidence of the Cartesian Product rule is calculated (Equation 2.6). First, the denominator value considers whether the body of the rule is true, whether or not the head of the rule is true. The rule is `(s, p, o')` $\wedge$ `(s', p, o)` $\Rightarrow$ `(s, p, o)`. Each triple `(s, p, o')` should be compared to each other triple in `p`, `(s, p, o')`. The head value, `(s, p, o)`, is created from the `s` and `o` values found above. This head will be stored in a set, because every head should only be counted once. It does not matter, at this point, if the head value `(s, p, o)` exists in the knowledge graph. By comparing each triple in `p` to each other triple in `p`, we get all possible head values.

The numerator of the confidence calculation is equal to support of the rule. We can compare the previously created set of head values where the body is true to the list of triples in `p` to find how many times the body and head of the rule are true. Any time the head from the dictionary matches a triple in `p`, the support is incremented.

Finally, the support is divided by the size of the set to get the confidence value. If the denominator is zero, as is the case if all `s` (or `o`) values are the same in `p`, the final confidence value will be zero. These special cases are technically Cartesian products, but are more accurately represented by the 1-to-Many (or Many-to-1) measurements seen below.

1-to-Many Each triple with predicate `p`, will be considered the head of the rule `(s, p, o')` $\Rightarrow$ `(s, p, o)`, and we look through the list of triples to find a `(s, p, o')` triple. Since the list of triples is sorted by `s` value, any triple with the same `s` value will be directly before (or after) the current triple. In the rare case that the previous (next) triple is also `(s, p, o)`, we keep looking backwards (forwards) at the next triple until we find one with a

different `o` value. If `(s, p, o')` is found, the support is incremented. For most triples, only the two adjacent triples must be examined. The head coverage is calculated using the support and size of the list of triples for each `p`.

Many-to-1 To find the head coverage of the Many-to-1 rule, the list of triples is sorted by `o` value instead of `s` value, and a similar algorithm to 1-to-Many is used, replacing `s` with `o` and vice-versa.

Chapter 7

Results

7.1 Original vs. Redundancy-Removed Knowledge Graphs

Previously, two knowledge graphs have been reworked to remove redundancy: WN18 [7] and FB15k [24]. The following figures show a comparison between WN18 and the redundancy-removed version WN18RR; and FB15k and the redundancy removed version FB15k-237. In these figures, we measure the distribution of redundancy with the "anomaly coefficient" measurement defined by Tiwari et al. [23] (as seen in Section 2.5), extended to include the additional redundancy types considered in this thesis. The rule-based redundancy measurements (as described in Table 4.1) were used as input to the anomaly coefficient equation. The anomaly coefficient takes the maximum redundancy value for each predicate. This allows a single redundancy type to be allocated to each predicate. This differs from the method described in this thesis, which aggregates all redundancy types to get the $\delta(p)$ value instead of taking a maximum to choose one type per predicate.

To display this information, the anomaly coefficient values were split into four equal categories between zero and one along the x-axis (values between 0 and 0.25 are in the first category, and so on). The percent of the triples in each of these categories is represented by the y-axis, with a label on top of each bar to indicate the percent value. Each bar is divided by color to show which type of redundancy it represents. This analysis was done for the predicates (labeled

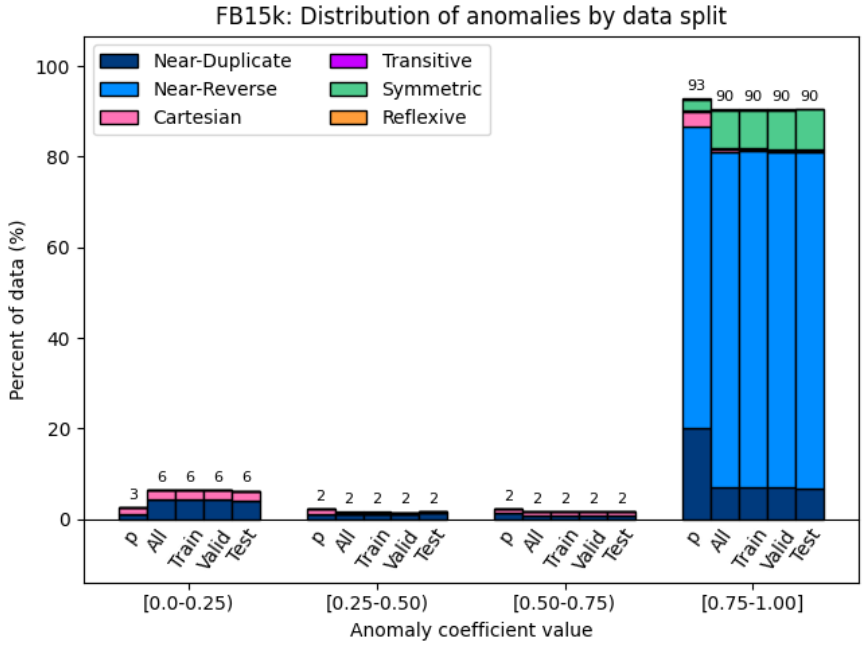

Figure 7.1: Distribution of different types of redundancies across different knowledge graph splits in FB15k.

'p'), the entire knowledge graph as a whole (labeled 'All'), and the Training, Validation, and Test splits of the knowledge graph. The bars labeled 'p' were calculated without considering how many triples are in each predicate, whereas the other bars do take triple count into effect. In Figure 7.1, we can see that FB15k as a whole ('All') has 90% of triples with an anomaly coefficient of 0.75 to 1.00, the highest redundancy category. We can determine that the majority of the triples are near-reverse, with some other triples having a majority of near-duplicate and symmetric redundancies.

Compare this to Figure 7.2 for FB15k-237, where only 22% of triples (looking at the 'All' bar) have an anomaly coefficient between 0.75 and 1.00. There is a much higher percent (48%) of triples in the lowest category, compared to only 6% in FB15k (Fig. 7.1). Among the redundancies that remain in the highest value range, we can see that all types of redundancies defined in this thesis are present, with transitive relations being most prevalent.

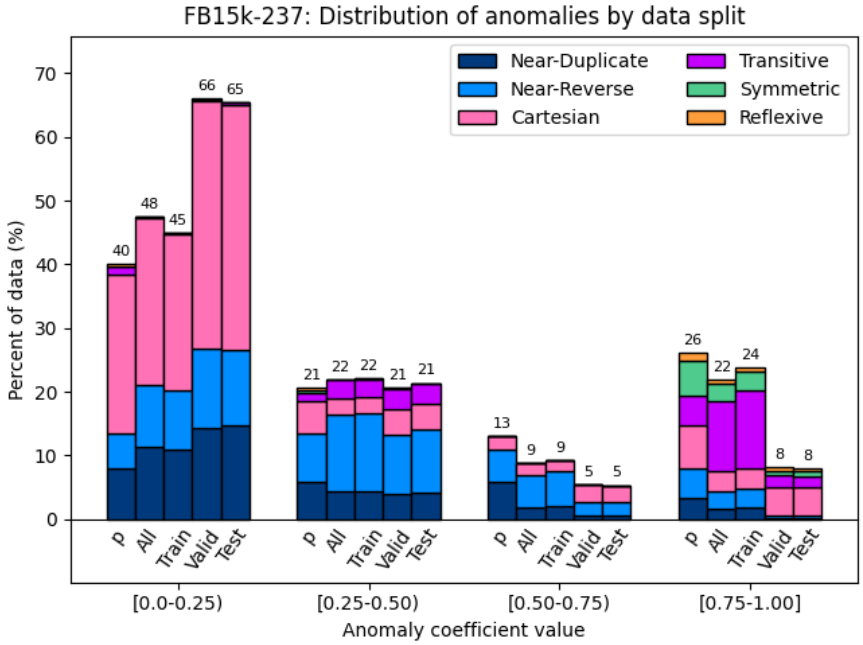

Figure 7.2: Distribution of different types of redundancies across different knowledge graph splits in FB15k-237.

Figure 7.2 also shows variation between the composition of the training, validation, and test sets. The validation and test sets have a much higher percentage (66% and 65% respectively) in the lowest redundancy range than the training set (45%). The validation and test sets also have much lower percentages of triples in the highest two redundancy ranges (8% and 8%), where the training set has a higher percentage (24%). Assuming the redundancies we have identified make link prediction easier, the training set will be much easier to predict due to its higher percentages of triples with high anomaly values. The validation and test sets would allegedly be more difficult to predict based on their higher percentage of triples with low anomaly values.

In the WN18 and WN18RR distributions presented in Figures 7.3 and 7.4, we see that the removal of redundancy from WN18 was not entirely successful. In WN18 (Fig. 7.3), 99% of all triples have a high anomaly coefficient value, with the other 1% being in the second highest redundancy category. Most of

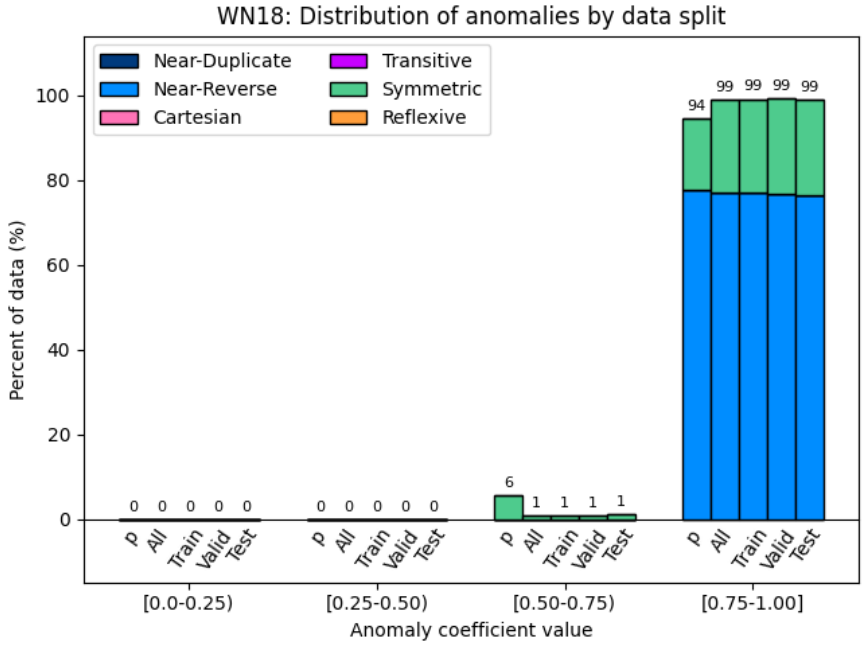

Figure 7.3: Distribution of different types of redundancies across different knowledge graph splits in WN18.

these are near-reverse triples, however there also are some symmetric triples. In Figure 7.4, we see that WN18RR still has 36% of all triples in the highest redundancy range, and they are all symmetric triples. The majority of the other triples (63%) are in the lowest redundancy range. This indicates that creators of WN18RR [7] only removed relations with redundancy in the near-reverse form and not the symmetric form.

7.2 Cartesian Product

Several other authors [2, 23] define Cartesian product measurements, but the measurements defined in this paper differ on edge cases, such as when a predicate contains only one o value for all triples. Handling relations with high connectivity but low inter-connectivity led to the largest differences between methods.

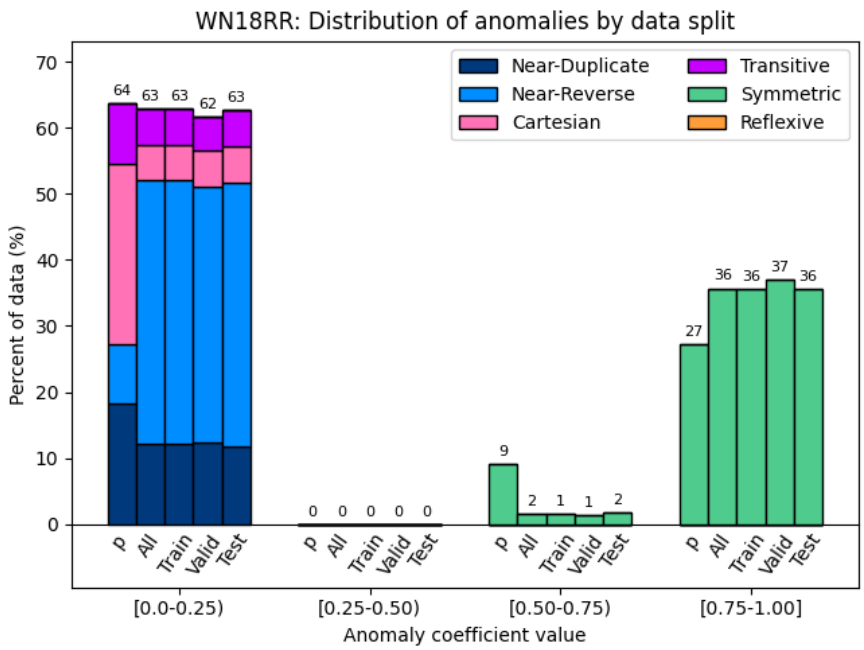

Figure 7.4: Distribution of different types of redundancies across different knowledge graph splits in WN18RR.

Several relations in FB15k-237 have only one object. Each subject in the relation maps to the same object. These relations have a Cartesian product anomaly coefficient of 1, because the number of triples is equal to the number of subjects multiplied by 1 (the number of objects). Take the toy knowledge graph

$$[(A, p, X), (B, p, X), (C, p, X), (D, p, X)]$$

for example. The Cartesian product anomaly coefficient as defined by [23] is 4/(4*1) = 1. In contrast, the Cartesian product confidence defined in this thesis is zero because no triples can match the body of the rule. The Cartesian product rule requires two triples that have different subjects and different objects. Using the example knowledge graph above, the body of the rule cannot be fulfilled because all of the triples have the same object. This means both the numerator (support) and denominator of the confidence calculation will be zero, which is manually converted to a zero value in the final results.

Another common pattern in FB15k-237 occurs when a relation has a very limited number of objects (or subjects), but all unique subjects (or objects). In a knowledge graph, this looks like multiple completely separate subgraphs. This leads to a Cartesian product anomaly coefficient as defined by [23] that is greater than zero, and a Cartesian product confidence of zero. An example of this is the toy knowledge graph

$$[(A, p, X), (B, p, X), (C, p, X), (D, p, Y), (E, p, Y), (F, p, Y)]$$

The anomaly coefficient [23] is calculated as follows: 6 triples / (6 subjects * 2 objects) = 0.5. The confidence is 0. While there are now triples that can make the body of the rule true, none of the heads that correspond with them exist in the knowledge graph. In other words, two disconnected triples that fit the body of the rule do exist, but there is no third triple (head) that ties them together. So the support (the numerator) is zero and the confidence is zero.

The reason that Cartesian confidence was chosen as the prevailing metric for measuring Cartesian product redundancies is because of these edge cases. In both cases where the confidence is zero, there is no inter-connectedness between the subjects and objects. In the first one, there is only one object, so there is no way to be inter-connected. In the second example, separate subgraphs are formed that do not interact with each other, so there is no evidence of Cartesian product. The second example is better represented by Many-to-1 cardinality than by Cartesian product redundancy.

7.3 Cardinality

The Cartesian product measurement we are using does not represent the pattern of few objects (subjects) and many subjects (objects). This pattern still has an impact on link prediction. To represent/measure this, we measure 1-to-Many and Many-to-1 predicates based off calculations of cardinality.

In FB15k-237, there are several relations that have Head Cardinality of 1 and Tail Cardinality of 0. These extreme values indicate that these relation have only one or a very limited number of tails (objects). For example, predicate `25` has 4364 triples and every single one has the same object. This is a very extreme case of Many-to-1. There are also relations with more than one, but a very limited number of objects, such as predicate `43` which has

4530 triples that contain only 2 different objects. The reason this relation had the extreme measurement of Head Cardinality = 1 and Tail Cardinality = 0 is because every triple has a unique subject, meaning no head maps to more than one tail. This is another extreme case of Many-to-1. This also means that within the predicates with very few objects and no overlapping subjects, there are completely separate subgraphs. Looking at the predicate name can give us a clue as to why this may occur. Predicate `43` represents `/people/person/gender`. Having only two objects with disconnected subgraphs makes sense in this context, since most databases consider only two mutually exclusive options for gender. Other examples represent incomplete or limited data, such as predicates involving currency which may only include a few objects due to the limited range of the data.

In a few cases the opposite is true, with Head Cardinality of 0 and Tail Cardinality of 1. This indicates a cardinality of 1-to-Many.

While these insights prove to be useful on cases where Head and Tail Cardinality values are extreme, the lack of a threshold makes determining the difference between "Many" and "1" difficult. For this reason, our results only look at some specific extreme instances where measuring cardinality is clear. We leave a more comprehensive analysis as future work.

7.4 Overall Redundancy Measurement

Table 7.1 shows the redundancy measurement for each knowledge graph with all types of redundancies combined: the overall redundancy measurement. The range of possible values for this measurement is $0.0 - 5.0$. Values are measured using the training set of each knowledge graph.

Notably, the highest redundancy seen among the knowledge graphs analyzed is 1.156 for FB15k. This is due to a large number of predicates with high redundancy. The highest value for an individual predicate is 4.145, meaning all five defined types of redundancy are prevalent in this predicate. FB15k-237, which is based on FB15k with many redundant predicates removed, has a much lower value of 0.671.

WN18 has the second highest value of the knowledge graphs analyzed, at 0.903. This is due to the large number of high near-reverse redundancy predicates in the knowledge graph. WN18RR in comparison has a much lower redundancy value at 0.360, due to the near-reverse redundancies being re-

Knowledge Graph	Redundancy
BioKG	0.761
FB13	0.135
FB15K	1.156
FB15K-237	0.671
Hetionet	0.301
NELL-995	0.371
WN11	0.903
WN18	0.943
WN18RR	0.360
YAGO3-10	0.631

Table 7.1: Overall redundancy value for each knowledge graph.

moved.

Interestingly, we also see higher values for BioKG, with an overall redundancy measurement of 0.761. This is higher than YAGO3-10, a known redundancy knowledge graph [2]. This is a new finding of this thesis.

Figure 7.5 breaks down which redundancy types contribute to the overall measurement. The values are calculated using Equation 4.7 where $G = G_{Train}$. Redundancy values $x(p)_i$ are calculated over the training split. Figure 7.5 shows these values, weighted by the number of triples in each predicate in the training split. Notably, the figure shows that all near-reverse redundancy has been removed from WN18RR, but symmetric redundancy remains. We can also see that the high redundancy in BioKG is coming from mainly transitive redundancies.

Figure 7.6 shows the redundancy values as they are distributed over the test set. The redundancy values are still measured over the training split, but they are weighted by the number of triples in each predicate in the test split, G_{Test}. The test split for each knowledge graph may not contain all predicates, so it is important to see how the overall redundancy changes when the composition of the data changes. Link prediction evaluation is completed on test set triples, so results will directly correlate to these values. We see that redundancy values for FB13, FB15k-237, and NELL-995 are significantly lower in the test split, meaning the most redundant predicates were not included in the test split. For the FB15k-237, this mirrors results seen when using the

anomaly coefficient in Figure 7.2.

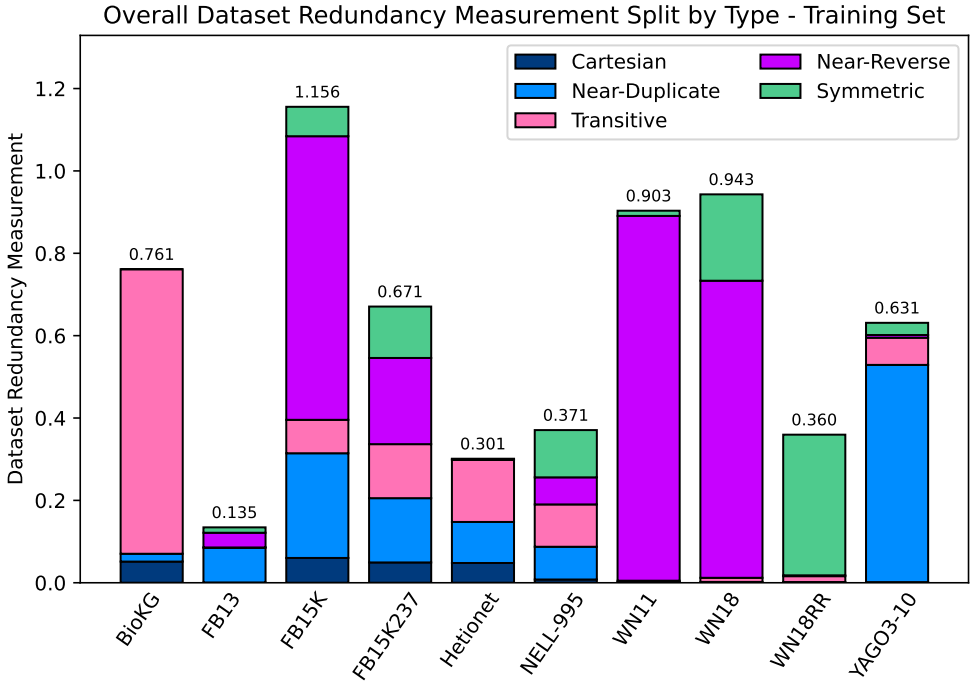

Figure 7.5: Overall redundancy value for the training split of each knowledge graph, grouped by how much each type of redundancy contributes to the final measurement.

7.5 Adjusting Link Prediction Evaluation with $\delta(p)$

Predicate-specific redundancy, $\delta(p)$, was measured for each predicate p in each knowledge graph. These measurements were normalized (see Equation 4.3) and used as weights when aggregating link prediction metrics (see Equation 4.5). The unweighted results of the link prediction evaluation using RR, R@1, R@10, and BPM@10 are in Table 7.2. The best result from the model with top performance is shown under the *Best* column, along with the error. Under the *Model Performance* column, we see the order in which each model performed.

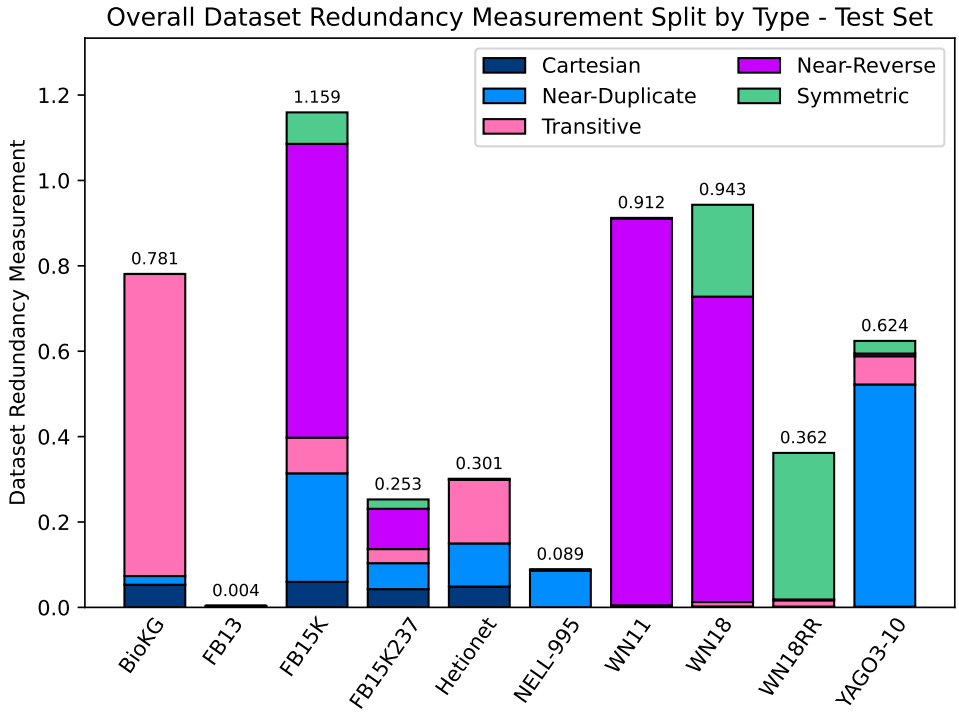

Figure 7.6: Overall redundancy value for the test split of each knowledge graph, grouped by how much of each type of redundancy is present in the test split.

These results are unweighted, meaning they are not altered in any way. These values reflect the capabilities of link prediction on the chosen benchmarking knowledge graphs without being adjusted for redundancies. Figure 7.7 shows the same unweighted values, for only the RR metric. Instead of only the best value for each metric, we see the values for all models and knowledge graphs for only one metric (RR). The result for one link prediction model on one benchmarking knowledge graph is represented by one block in the grid, with the value corresponding to the color of the block.

In Figure 7.7, we can see that most link prediction models perform very well on WN18, with higher RR values than any other knowledge graph. Similarly in Table 7.2 we see that link prediction scores for WN18 are the highest

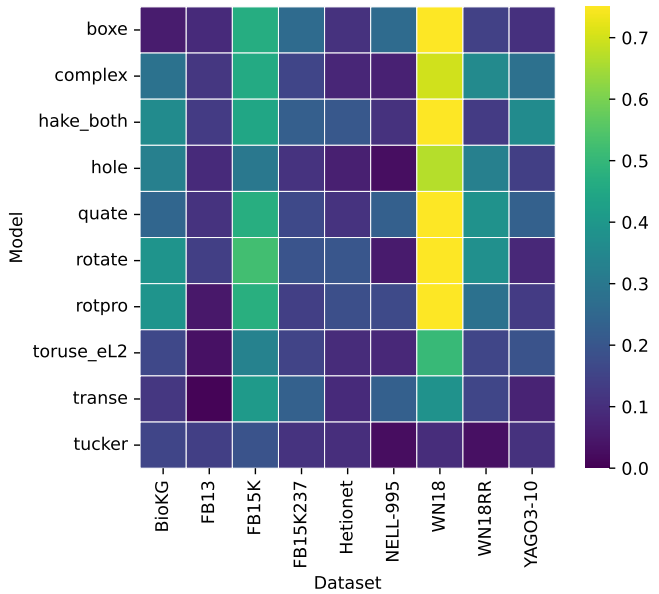

Figure 7.7: Unweighted link prediction results with the RR metric.

among all the knowledge graphs for all listed metrics. The results for FB15k are also high, as well as the results for BioKG. Out of the knowledge graphs analyzed, these three have the highest overall redundancy values in both their training and test sets. The knowledge graph with the lowest overall redundancy, FB13, has the overall lowest link prediction performance. All of these observations seem to support the hypothesis that redundancies result in better link prediction scores. We can also see the performance decrease from FB15k to FB15k-237 and from WN18 to WN18RR. This phenomenon is well recorded across several metrics. This seems to support the aforementioned hypothesis as well. We see knowledge graphs that are supposed to contain similar data, just without redundancies, but link prediction does not perform as well on them. Interestingly, WN18RR, which has an overall redundancy measurement of 0.360, performs similarly to BioKG which has an overall redundancy measurement of 0.761.

In order to adjust these link prediction results for the redundancy of each benchmarking knowledge graph, we apply $\delta(p)$ as a weight. The weighted

Table 7.2: **Unweighted link prediction results**. Model names are abbreviated as follows: Bo=BoxE, Co=ComplEx, HA=HAKE, Ho=HolE, Qu=QuatE, Rt=RotatE, RP=RotPro, To=TorusE, Tr=TransE, Tu=TuckER.

	Metric	Best	Model Performance
FB13	RR	0.140±0.301	$Rt > Tu > HA > Co > Qu > Bo > Ho > RP > To > Tr$
	$R@1$	0.106±0.308	$Tu > Rt > HA > Co > Qu > Bo > RP > Ho > To > Tr$
	$R@10$	0.230±0.421	$HA > Rt > Co > Qu = Tu > Ho > Bo > To > RP > Tr$
	$BPM@10$	0.658±0.237	$Tu > Rt > Co > Bo > Qu > RP > To > Ho > Tr > HA$
FB15k	RR	0.523±0.416	$Rt > RP > Qu = Bo > Co > HA > Tr > To > Ho > Tu$
	$R@1$	0.397±0.489	$Rt > Qu > Bo > RP > Co > HA > Tr > Ho > To > Tu$
	$R@10$	0.736±0.441	$Rt > HA > RP > Bo > Co > Qu > Tr > To > Ho > Tu$
	$BPM@10$	0.731±0.237	$Rt > Tu > Qu > Bo > RP > Co > HA > Tr > Ho > To$
FB15k-237	RR	0.260±0.362	$Bo > Tr = HA > Rt > Qu > Co > To > RP > Ho = Tu$
	$R@1$	0.169±0.375	$Bo > Tr > HA > Rt > Qu > Co > RP > To > Ho > Tu$
	$R@10$	0.445±0.497	$Bo > Tr = HA > Rt > To > Qu > Co > RP > Tu > Ho$
	$BPM@10$	0.701±0.176	$Bo > Tr > Tu > Rt > Qu > Co > To > RP > HA > Ho$
WN18	RR	0.889±0.303	$Qu > Bo = RP = Rt > HA > Co > Ho > To > Tr > Tu$
	$R@1$	0.879±0.326	$Qu > Rt = Bo > RP > HA > Co > Ho > To > Tu > Tr$
	$R@10$	0.931±0.253	$RP > Tr = Qu = Bo > To = Rt > HA > Ho = Co > Tu$
	$BPM@10$	0.936±0.182	$Qu > Bo = RP > Rt > HA > Co > Ho > To > Tu > Tr$
WN18RR	RR	0.379±0.478	$Qu > Rt > Co > Ho > RP > To = Tr > Bo = HA > Tu$
	$R@1$	0.368±0.482	$Qu > Rt > Co > Ho > RP > Bo > HA > Tu > Tr = To$
	$R@10$	0.435±0.496	$Tr > RP > Qu = To = Rt = Co > Ho > Bo = HA > Tu$
	$BPM@10$	0.683±0.259	$Qu > Co > Ho > Bo > Rt > RP > To > Tr > Tu = HA$
BioKG	RR	0.389±0.379	$Rt = RP > HA > Ho > Co > Qu > To > Tu > Tr > Bo$
	$R@1$	0.244±0.430	$RP = HA > Rt = Ho > Co > Qu > Tu > Tr = To > Bo$
	$R@10$	0.649±0.477	$Rt > RP > HA > Ho > Co > Qu > To > Tu > Tr > Bo$
	$BPM@10$	0.691±0.222	$Rt = RP > HA > Ho > Co > Qu > To > Tr > Bo > Tu$
Hetionet	RR	0.205±0.341	$HA = Rt > RP > Qu = Bo = Tu > To > Tr = Co > Ho$
	$R@1$	0.138±0.344	$HA = Rt > RP > Qu = Bo > Tu = Tr = To > Co > Ho$
	$R@10$	0.342±0.475	$Rt = HA > RP > Qu = Bo = Tu = To > Tr = Co > Ho$
	$BPM@10$	0.690±0.172	$HA = Rt > RP > Bo = Tr > Tu > To > Ho > Qu > Co$
NELL-995	RR	0.258±0.392	$Bo > Qu = Tr > RP > HA = To = Co > Rt > Ho > Tu$
	$R@1$	0.201±0.401	$Bo > Qu = Tr > RP > HA > To = Rt > Ho = Tu > Co$
	$R@10$	0.371±0.483	$Bo > Qu = Tr > RP > Co > HA = To > Rt > Ho > Tu$
	$BPM@10$	0.728±0.168	$Bo > Tr > To > Qu > RP = Rt > HA > Tu > Ho > Co$
YAGO3-10	RR	0.359±0.389	$HA > Co > Qu > To > Ho > RP > Tu > Bo > Rt > Tr$
	$R@1$	0.224±0.417	$HA > Co > Qu > To > RP = Ho > Tu > Bo > Rt = Tr$
	$R@10$	0.573±0.495	$HA > Co > Qu > To > Ho > RP > Bo > Tu > Rt > Tr$
	$BPM@10$	0.721±0.114	$Tr > Bo > Co = Tu = To > Ho > HA > Qu > Rt = RP$

results for several metrics can be seen in Table 7.3. The weighted RR results are seen in Figure 7.8. For a more intuitive view of the results, instead of seeing the values of the weighted RR metric like we see in the table, Figure 7.8 tracks gain: the difference between the unweighted and weighted RR results.

$$\text{Gain} = \text{Unweighted} - \text{Weighted} \tag{7.1}$$

The positive values, in red, indicate that the weighted values were lower than unweighted values. This is the result expected with our hypothesis: weighting each predicate for redundancy resulted in a decreased link prediction evaluation score. The negative values, in blue, indicate that weighted values were higher than unweighted values, indicating the opposite result. Notably, we see WN18RR weighted values are much lower than unweighted values (dark red color). From the tables we see for WN18RR using the RR metric and the QuatE model, unweighted is 0.379 and weighted is 0.167. The gain is 0.212, shown by the dark red value at the intersection of *quate* and *WN18RR* in Figure 7.8. This implies that redundancies have a severe impact on the link prediction of WN18RR triples. In fact, we can see from the figure that this is the most severe impact: no other knowledge graph has values this high (or red squares this dark). This is not the expected result, we would expect the highest positive gain to be associated with the knowledge graph with the most redundancy. WN18RR has an overall redundancy measurement of only 0.360. Something to consider is that WN18RR has 3 (out of 11) predicates with a symmetric redundancy value of 1.0, the highest possible value for one type of redundancy. All other predicates have $\delta(p) \leq 0.05$, with the exception of predicate `3`: $\delta(3) = 0.90$. We can assume the severity of redundancy in certain predicates is what caused the extreme gain.

We do see positive gain with all models for FB15k, and with most models for BioKG and YAGO3-10. Each of these knowledge graphs have high overall redundancy. This means that link prediction performance on these knowledge graphs is being impacted by redundancy in a negative way. However we see almost zero gain for WN18, implying that redundancies do not impact link prediction much. Looking at the tables, we can see the gain is slightly positive, but not to the level we would expect for the knowledge graph with the second-highest redundancy.

Surprisingly, we see negative results for Hetionet, indicating that weighted results were higher than unweighted results for some models. The gain of the

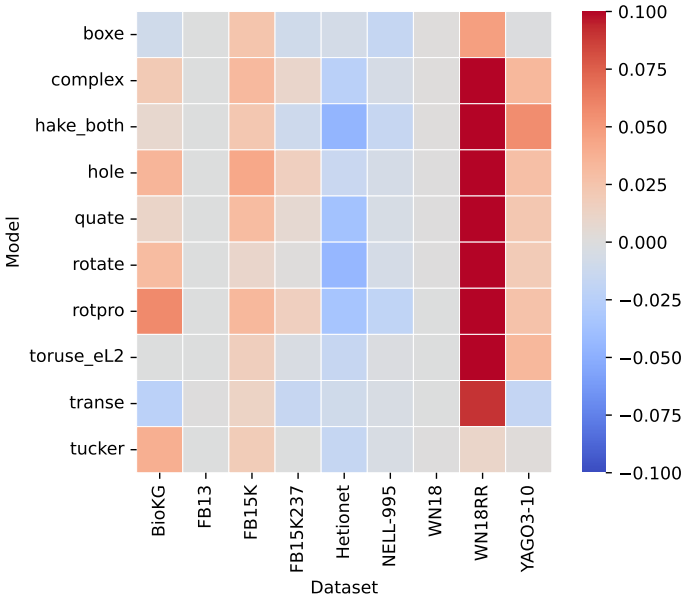

Figure 7.8: Gain of weighted link prediction results with the RR metric.

top-performing model (HAKE) for Hetionet with the RR metric is -0.041. Nothing about Hetionet's composition immediately indicates the negative result: it has a fairly low overall redundancy measurement of 0.301, composed of a few triples with high transitive or near-duplicate redundancy, and several triples with almost zero redundancy. This is not entirely dissimilar from the composition of WN18RR, which has very high positive gain. The negative gain contradicts the hypothesis that redundancies only increase link prediction performance. The combination of negative and positive gain shows that redundancies can have different impacts on different knowledge graphs.

Using other metrics for analysis shows further mixed results. Figures 7.9 and 7.10 show the gain of link prediction results using the BPM@10 and SetR respectively. These results differ from the previously seen RR metric (Figure 7.8) in certain cases. For example, BMP@10 shows mainly negative values for FB15k-237, whereas SetR shows all positive values. BPM@10 shows positive values for Tucker on NELL-995 whereas RR and SetR show negative results. This further supports the idea that redundancies have varied impacts using

Table 7.3: **Link prediction results weighted with predicate-specific redundancy measurements, $\delta(p)$.** Model names are abbreviated as follows: Bo=BoxE, Co=ComplEx, HA=HAKE, Ho=HolE, Qu=QuatE, Rt=RotatE, RP=RotPro, To=TorusE, Tr=TransE, Tu=TuckER.

	Metric	Best	Model Performance
FB13	RR	0.158±0.396	$Tu > Rt = HA > Co > Qu > Bo > Ho > RP > To > Tr$
	$R@1$	0.124±0.384	$Tu > Rt > HA > Bo > Co > Qu > RP > Ho > To > Tr$
	$R@10$	0.231±0.491	$HA > Rt > Co > Tu = Qu > Ho > Bo > To > RP > Tr$
	$BPM@10$	0.682±0.444	$Tu > Rt > Bo > Co > Qu > RP > To > Ho > HA > Tr$
FB15K	RR	0.477±0.438	$Rt > Bo > RP > Qu > HA > Co > Tr > To > Ho > Tu$
	$R@1$	0.373±0.490	$Rt > Bo > Qu > RP > HA > Co > Tr > To > Ho > Tu$
	$R@10$	0.664±0.518	$Rt > HA > Bo > RP > Tr > Co > Qu > To > Ho > Tu$
	$BPM@10$	0.718±0.457	$Tu > Rt > Bo > Qu > Co > RP > Tr > HA > To > Ho$
FB15K237	RR	0.273±0.450	$Bo > Tr = HA > Rt > Qu > To > Co > RP > Tu = Ho$
	$R@1$	0.186±0.454	$Bo > Tr > HA > Rt > Qu > Co > RP > To > Tu > Ho$
	$R@10$	0.448±0.611	$Bo > Tr = HA > Rt > To > Qu > Co > RP > Tu > Ho$
	$BPM@10$	0.713±0.405	$Bo > Tr > Tu > Rt > Qu > Co > To > HA > RP > Ho$
WN18	RR	0.868±0.529	$Qu > RP = Bo > Rt > HA > Co > Ho > To > Tr > Tu$
	$R@1$	0.855±0.543	$Qu > Rt > RP > Bo > HA > Co > Ho > To > Tr > Tu$
	$R@10$	0.923±0.521	$RP > Tr = Bo = Qu > To = Rt > HA > Ho = Co > Tu$
	$BPM@10$	0.919±0.473	$Qu > RP = Bo > Rt > HA > Co > Ho > To > Tu > Tr$
WN18RR	RR	0.167±0.253	$Qu > Rt > Co > Ho > RP > Bo > Tr = To > HA > Tu$
	$R@1$	0.153±0.250	$Qu > Rt > Co > Ho > RP > Bo > HA > Tu > Tr = To$
	$R@10$	0.246±0.393	$Tr > RP > To = Qu > Rt = Co > Ho > Bo = HA > Tu$
	$BPM@10$	0.630±0.364	$Bo > Qu > Co > Ho > RP > Rt > To > Tu > Tr > HA$
BioKG	RR	0.354±0.448	$Rt > HA > RP > Ho > Co > Qu > To > Tr > Tu > Bo$
	$R@1$	0.251±0.511	$HA > Rt > RP > Ho > Co > Qu > Tr = To > Tu > Bo$
	$R@10$	0.568±0.534	$Rt > HA > RP > Ho > Co > Qu > To > Tr > Tu > Bo$
	$BPM@10$	0.701±0.460	$Rt > HA > RP > Qu > Co > Ho > Tr > To > Bo > Tu$
Hetionet	RR	0.246±0.442	$HA = Rt > RP > Qu > Tu > Co > Bo > To > Tr > Ho$
	$R@1$	0.169±0.442	$HA = Rt > RP > Qu > Tu > Bo > To > Co > Tr > Ho$
	$R@10$	0.400±0.603	$Rt = HA > RP > Qu > Tu > Co > To > Bo > Tr > Ho$
	$BPM@10$	0.732±0.443	$HA = Rt > RP > Bo = Tr > Tu > To > Ho > Qu > Co$
NELL-995	RR	0.287±0.502	$Bo > Qu = Tr > RP > HA = To = Co > Rt > Ho > Tu$
	$R@1$	0.226±0.506	$Bo > Qu = Tr > RP > HA > To = Rt > Ho = Tu > Co$
	$R@10$	0.413±0.619	$Bo > Tr = Qu > RP > Co > HA = To > Rt > Ho > Tu$
	$BPM@10$	0.742±0.430	$Bo > Tr > Qu > To > RP = Rt > HA > Ho > Co = Tu$
YAGO3-10	RR	0.282±0.352	$HA > Co > Qu > To > Bo > Tu > Ho > Tr > RP > Rt$
	$R@1$	0.179±0.367	$HA > Co > Qu > To > Tu > Tr > Bo > RP = Ho > Rt$
	$R@10$	0.445±0.454	$HA > Co > Qu > To > Bo > Ho > Tu > RP > Tr > Rt$
	$BPM@10$	0.726±0.407	$Tr > Bo > Tu = Co = To > Ho > Qu > HA > Rt = RP$

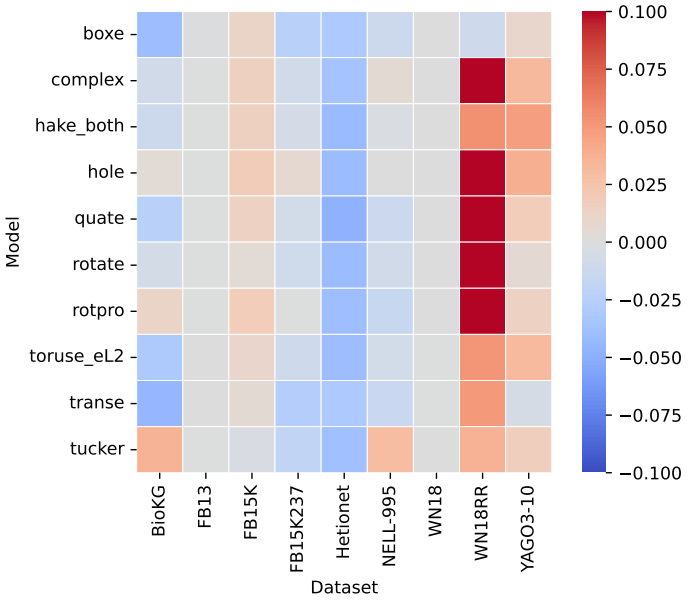

Figure 7.9: Gain of weighted link prediction results with the BPM@10 metric.

different metrics, models, and link prediction metrics.

Based on the above analysis, the hypothesis that redundancies in benchmarking knowledge graphs make link prediction less challenging and result in better scores on link prediction evaluation metrics is not proven to always be correct. This contradicts previous work, which is based on the tenet that redundancies are having negative impact on link prediction evaluation. Given this result, it follows that the second hypothesis is correct: removing redundancy from benchmarking knowledge graphs is leading to unrealistic link prediction evaluation. Each benchmarking knowledge graph with redundancy removed displays overall worse link prediction performance than the original knowledge graph. If redundancies cannot be proven to be the only reason for the performance difference, then it is the removal of facts from the graph that is creating a drop in performance. FB15k-237 and WN18RR are supposed to be representative of FB15k and WN18, just without the most redundant predicates. The decreased link prediction performance on the knowledge graphs with redundancy removed implies that this method is indeed flawed,

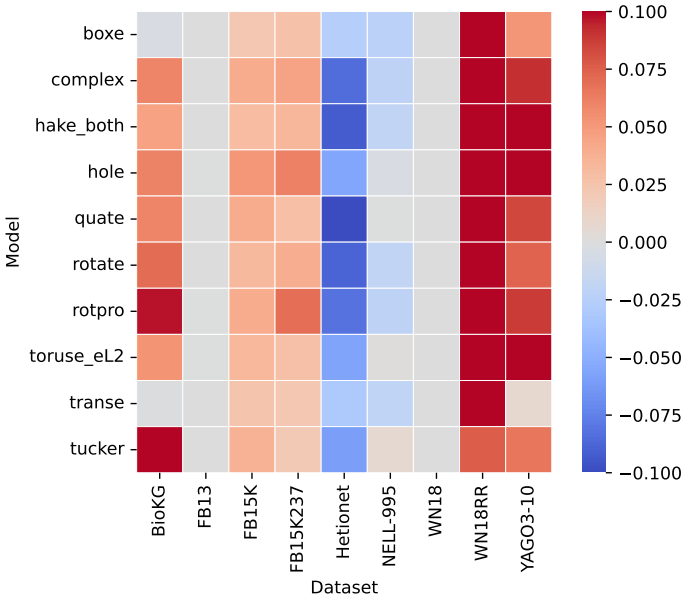

Figure 7.10: Gain of weighted link prediction results with the SetR metric.

and should not be used as a solution for redundant benchmarking knowledge graphs.

Chapter 8

Conclusion

The idea of redundancy in benchmarking knowledge graphs has existed in the literature since it was introduced by Toutanova and Chen in 2015 [24]. They introduced the problem that redundancies might present: a simple, feature-based model that only predicted near-duplicate and near-reverse triples performed better than state-of-the-art link prediction embedding models. They removed triples past a certain threshold of redundancy from FB15k to create FB15k-237, and similarly Dettmers et al. [7] removed redundancy from WN18 to create WN18RR. When link prediction models could not perform as well on these new knowledge graphs, it was assumed as confirmation that redundancies caused (artificially) high link prediction scores. This thesis presents a new way of thinking about this issue.

First, we hypothesized, similar to previous researchers, that redundancies in benchmarking knowledge graphs make link prediction less challenging, which results in better scores on link prediction evaluation metrics. Using the redundancy values for each predicate in each knowledge graph, $\delta(p)$, we weighted link prediction results. We expected that applying negative weights would decrease link prediction scores across the board, but it had mixed results: some link prediction scores decreased (like FB15k and WN18RR), but some increased (like Hetionet). More so, the mixed results were seen for the same knowledge graph across different link prediction models and evaluation metrics. The overall amount of redundancy in a knowledge graph did not directly correlate to the gain of weighted link prediction results. This refutes the hypothesis that redundancies have a strictly amplifying effect on link pre-

diction performance.

We doubted the legitimacy of the methods presented by previous work for removing redundancies in knowledge graphs. How can link prediction be expected to perform well with half the facts missing? This question is represented by our second hypothesis: removing facts to create a new benchmarking knowledge graph without redundancies is a flawed method because it removes true facts that belong in the graph, which leads to unrealistic link prediction evaluation. As we see above, redundancies do not have a consistent impact on link prediction, so they cannot be blamed for the dip in link prediction performance on knowledge graphs with redundancies removed. We assert that removing redundancies from knowledge graphs in the methods previously seen is harmful to link prediction, as it disconnects the data in the knowledge graph. Benchmarking knowledge graphs with redundancy removed, such as FB15k-237, WN18RR, and more, are often used in place of FB15k, WN18, etc. as a solution to the uncertainty that redundancies present. We assert that these knowledge graphs should not be used as solutions, and researchers should proceed with caution using them. Being aware of the redundancies in each knowledge graph and the unpredictable effect they may cause on link prediction results is imperative to anyone studying these fields.

This thesis introduces methods for measuring redundancy at several levels: individual types of redundancy, individual predicates, and entire knowledge graphs. The rule-based methodology used to measure redundancy allows for the addition of new rules for other types of redundancies. Already the definitions for cardinality-based redundancy have been introduced in Chapter 5. Future work could expand on this with more in-depth experiments that allow the true impact of cardinality to be studied. Other redundancies could include functionality or the Z-paradox, which were introduced in the related work in Chapter 3.

In this thesis, link prediction results were adjusted with the redundancy value of each individual predicate, however this value, $\delta(p)$, was a sum of several types of redundancy. Further studies could analyze the different types of redundancy individually. This could show which redundancies are most impactful, or even the explain mixed results seen in this thesis.

More than anything, this thesis provides a framework for measuring redundancies and an analysis of several commonly-studied types of redundancy in ten of the most frequently used benchmarking knowledge graphs. Others

are encouraged to use these methods for other types of redundancy or other benchmarking knowledge graphs to expand this study in new ways.

Bibliography

[1] Ralph Abboud, İsmail İlkan Ceylan, Thomas Lukasiewicz, and Tommaso Salvatori. BoxE: A box embedding model for knowledge base completion. In *NeurIPS*, 2020.

[2] Farahnaz Akrami, Mohammed Samiul Saeef, Qingheng Zhang, Wei Hu, and Chengkai Li. Realistic re-evaluation of knowledge graph completion methods: An experimental study. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, SIGMOD '20, page 1995–2010. Association for Computing Machinery, 2020.

[3] Leif Azzopardi, Paul Thomas, and Alistair Moffat. cwl_eval: An evaluation tool for information retrieval. In *SIGIR*, pages 1321–1324, 2019.

[4] Ivana Balazevic, Carl Allen, and Timothy M. Hospedales. TuckER: Tensor factorization for knowledge graph completion. In *EMNLP-IJCNLP*, pages 5184–5193, 2019.

[5] Antoine Bordes, Xavier Glorot, Jason Weston, and Yoshua Bengio. A semantic matching energy function for learning with multi-relational data. *Machine Learning*, 2013.

[6] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Durán, Jason Weston, and Oksana Yakhnenko. Translating embeddings for modeling multi-relational data. In *Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2*, NIPS'13, page 2787–2795, 2013.

[7] Tim Dettmers, Pasquale Minervini, Pontus Stenetorp, and Sebastian Riedel. Convolutional 2D knowledge graph embeddings. In *Proceed-*

ings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI'18/IAAI'18/EAAI'18, page 1811–1818. AAAI Press, 2018.

[8] Zehua Duo, Jiang Li, Xiangdong Su, and Guanglai Gao. Flore: Integrating full lorentz group and directional offsets for effective knowledge graph embedding. In *AAAI*, 2026.

[9] Takuma Ebisu and Ryutaro Ichise. TorusE: Knowledge graph embedding on a lie group. In *AAAI*, pages 1819–1826, 2018.

[10] Norbert Fuhr. Some common mistakes in ir evaluation, and how they can be avoided. page 32–41, 2018.

[11] Luis Antonio Galárraga, Christina Teflioudi, Katja Hose, and Fabian Suchanek. AMIE: association rule mining under incomplete evidence in ontological knowledge bases. In *Proceedings of the 22nd International Conference on World Wide Web*, WWW '13, page 413–422. Association for Computing Machinery, 2013.

[12] Daniel Scott Himmelstein, Antoine Lizee, Christine Hessler, Leo Brueggeman, Sabrina L Chen, Dexter Hadley, Ari Green, Pouya Khankhanian, and Sergio E Baranzini. Systematic integration of biomedical knowledge prioritizes drugs for repurposing. *Elife*, 6:e26726, 2017.

[13] Aidan Hogan, Eva Blomqvist, Michael Cochez, Claudia D'amato, Gerard De Melo, Claudio Gutierrez, Sabrina Kirrane, José Emilio Labra Gayo, Roberto Navigli, Sebastian Neumaier, Axel-Cyrille Ngonga Ngomo, Axel Polleres, Sabbir M. Rashid, Anisa Rula, Lukas Schmelzeisen, Juan Sequeda, Steffen Staab, and Antoine Zimmermann. Knowledge graphs. *ACM Comput. Surv.*, 54(4), 2021.

[14] Narayanan Asuri Krishnan and Carlos R. Rivero. A method for assessing inference patterns captured by embedding models in knowledge graphs. In *WWW*, pages 2030–2041, 2024.

[15] Farzaneh Mahdisoltani, Joanna Biega, and Fabian M. Suchanek. YAGO3: A knowledge base from multilingual wikipedias. In *CIDR*, 2015.

[16] Maximilian Nickel, Lorenzo Rosasco, and Tomaso A. Poggio. Holographic embeddings of knowledge graphs. In *AAAI*, pages 1955–1961, 2016.

[17] Ciyuan Peng, Feng Xia, Mehdi Naseriparsa, and Francesco Osborne. Knowledge graphs: Opportunities and challenges. *Artif Intell Rev 56*, page 13071–13102, 2023.

[18] Aishwarya Rao, Narayanan Asuri Krishnan, and Carlos R. Rivero. Using model calibration to evaluate link prediction in knowledge graphs. In *WWW*, pages 2042–2051, 2024.

[19] Andrea Rossi, Denilson Barbosa, Donatella Firmani, Antonio Matinata, and Paolo Merialdo. Knowledge graph embedding for link prediction: A comparative analysis. *ACM Trans. Knowl. Discov. Data*, 15(2), 2021.

[20] Richard Socher, Danqi Chen, Christopher D. Manning, and Andrew Y. Ng. Reasoning with neural tensor networks for knowledge base completion. In *NIPS*, pages 926–934, 2013.

[21] Tengwei Song, Jie Luo, and Lei Huang. Rot-pro: Modeling transitivity by projection in knowledge graph embedding. In *NeurIPS*, pages 24695–24706, 2021.

[22] Zhiqing Sun, Zhi-Hong Deng, Jian-Yun Nie, and Jian Tang. RotatE: Knowledge graph embedding by relational rotation in complex space. In *ICLR*, 2019.

[23] Sudhanshu Tiwari, Iti Bansal, and Carlos R. Rivero. Revisiting the evaluation protocol of knowledge graph completion methods for link prediction. In *Proceedings of the Web Conference 2021*, WWW '21, page 809–820. Association for Computing Machinery, 2021.

[24] Kristina Toutanova and Danqi Chen. Observed versus latent features for knowledge base and text inference. In *Proceedings of the 3rd Workshop on Continuous Vector Space Models and their Compositionality*, page 57–66. Association for Computational Linguistics, 2015.

[25] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. Complex embeddings for simple link prediction. In *ICML*, volume 48, pages 2071–2080, 2016.

[26] Brian Walsh, Sameh K. Mohamed, and Vít Novácek. BioKG: A knowledge graph for relational learning on biological data. In *CIKM*, pages 3173–3180, 2020.

[27] Wenhan Xiong, Thien Hoang, and William Yang Wang. DeepPath: A reinforcement learning method for knowledge graph reasoning. In *EMNLP*, pages 564–573, 2017.

[28] Fan Zhang, Yiqun Liu, Xin Li, Min Zhang, Yinghui Xu, and Shaoping Ma. Evaluating web search with a bejeweled player model. In *SIGIR*, pages 425–434, 2017.

[29] Shuai Zhang, Yi Tay, Lina Yao, and Qi Liu. Quaternion knowledge graph embeddings. In *NeurIPS*, pages 2731–2741, 2019.

[30] Zhanqiu Zhang, Jianyu Cai, Yongdong Zhang, and Jie Wang. Learning hierarchy-aware knowledge graph embeddings for link prediction. In *AAAI*, pages 3065–3072, 2020.

Appendices

Appendix A

Implementation Appendix

A.1 Cypher Queries

To supplement the Java implementation of measuring redundancies, several Cypher queries were completed on Neo4j editions of certain databases. A sampling of predicates and redundancy types were checked with Cypher queries and the results matched the Java implementation. Below are selected examples of queries performed on WN18.

```
MATCH ()-[ p:`_verb_group` ]->()
RETURN COUNT(p) as cnt
```

Figure A.1: Counting the number of triples with the predicate `_verb_group`

A.1.1 Cartesian product redundancies with Cypher

Each Cypher query is generated from a rule given as input text. The rule for Cartesian product redundancies for the `_verb_group` predicate is represented `_verb_group(?x, ?y) <= _verb_group(?x, ?a) AND _verb_group(?b, ?y)`. The head of the rule, represented in section 4.1 as `(s, p, o)`, uses the format `p(?x, ?y)` in the implementation. Other variables such as `s'` and `o'` are represented by fresh variables such as `a`, `b` and `z`.

Measuring the support of a rule requires finding the `(s, o)` or `(x, y)` pairs

where the head and body of the rule are true. The Cypher query in Figure A.2 represents this. Head coverage is then found by dividing the support measurement by the number of triples in `p` (Figure A.1).

```
MATCH (x)-[:`_verb_group` ]->(y),
(x)-[:`_verb_group` ]->(a),
(b)-[:`_verb_group` ]->(y)
WITH DISTINCT [x.id, y.id] AS pairs
RETURN COUNT(pairs) AS cnt
```

Figure A.2: Measuring the support of the Cartesian product rule

Measuring confidence requires dividing the support measurement by the number of times the body of a rule is true. This query finds that value.

```
MATCH (x)-[:`_verb_group` ]->(a),
(b)-[:`_verb_group` ]->(y)
WITH DISTINCT [x.id, y.id] AS pairs
RETURN COUNT(pairs) AS cnt
```

Figure A.3: Measuring the denominator of confidence for Cartesian product